

CHAPTER NO.	CHAPTER NAME	PAGE NO.
1	INTRODUCTION TO PYTHON	2
2	PYTHON FUNDAMENTALS	5
3	DATA HANDLING	16
4	FLOW OF CONTROL	27
5	FUNCTIONS IN PYTHON	38
6	STRING IN PYTHON	50
7	LIST IN PYTHON	61
8	TUPLE IN PYTHON	75
9	DICTIONARY IN PYTHON	85
10	SORTING	95
11	DEBUGGING PROGRAMS	99
12	EXPLANATION OF KEYWORDS	103

By: Vikash Kumar Yadav

PGT-Computer Science

K.V. No.-IV ONGC Vadodara

CHAPTER-1

INTRODUCTION TO PYTHON

1.1 Introduction:

- General-purpose Object Oriented Programming language.
- High-level language
- Developed in late 1980 by **Guido van Rossum** at National Research Institute for Mathematics and Computer Science in the Netherlands.
- It is derived from programming languages such as ABC, Modula 3, small talk, Algol-68.
- It is Open Source Scripting language.
- It is **Case-sensitive language** (Difference between uppercase and lowercase letters).
- One of the official languages at Google.

1.2 Characteristics of Python:

- ✓ Interpreted: Python source code is compiled to byte code as a **.pyc** file, and this byte code can be interpreted by the interpreter.
- ✓ Interactive
- ✓ Object Oriented Programming Language
- ✓ Easy & Simple
- ✓ Portable
- ✓ Scalable: Provides improved structure for supporting large programs.
- ✓ Integrated
- ✓ Expressive Language

1.3 Python Interpreter:

Names of some Python interpreters are:

- PyCharm
- Python IDLE
- The Python Bundle
- pyGUI
- Sublime Text etc.

There are **two modes** to use the python interpreter:

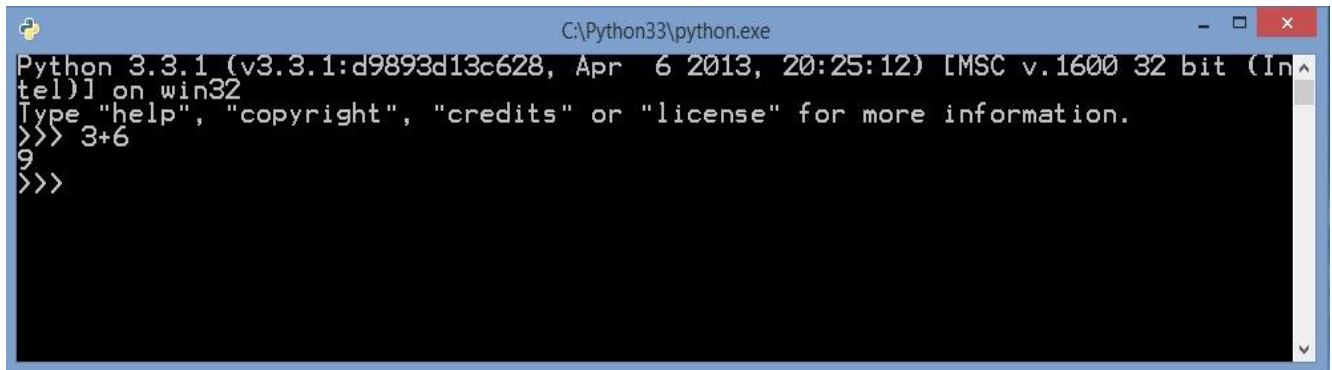
- i. Interactive Mode
- ii. Script Mode

- i. **Interactive Mode:** Without passing python script file to the interpreter, directly execute code to Python (Command line).

Example:

```
>>>6+3
```

Output: 9



```
C:\Python33\python.exe
Python 3.3.1 (v3.3.1:d9893d13c628, Apr 6 2013, 20:25:12) [MSC v.1600 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+6
9
>>>
```

Fig: Interactive Mode

Note: >>> is a command the python interpreter uses to indicate that it is ready. The interactive mode is better when a programmer deals with small pieces of code.

To run a python file on command line:

```
exec(open("C:\Python33\python programs\program1.py").read( ))
```

- ii. **Script Mode:** In this mode source code is stored in a file with the **.py** extension and use the interpreter to execute the contents of the file. To execute the script by the interpreter, you have to tell the interpreter the name of the file.

Example:

if you have a file name **Demo.py** , to run the script you have to follow the following steps:

Step-1: Open the text editor i.e. Notepad

Step-2: Write the python code and save the file with .py file extension. (Default directory is *C:\Python33/Demo.py*)

Step-3: Open IDLE (Python GUI) python shell

Step-4: Click on file menu and select the open option

Step-5: Select the existing python file

Step-6: Now a window of python file will be opened

Step-7: Click on Run menu and the option Run Module.

Step-8: Output will be displayed on python shell window.

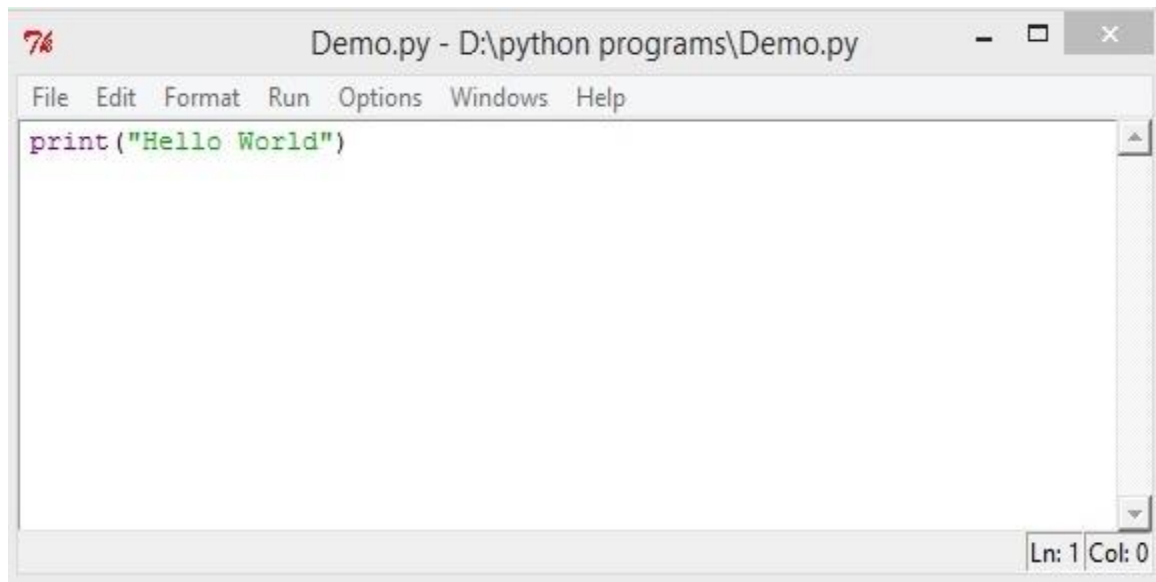


Fig. : IDLE (Python GUI)

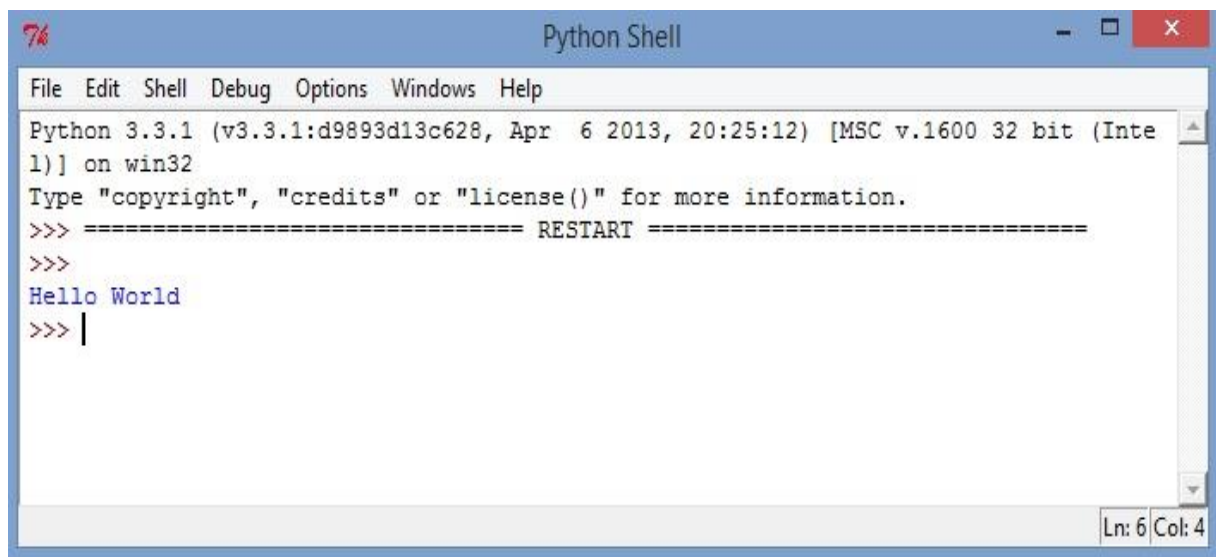


Fig: Python Shell

CHAPTER-2

PYTHON FUNDAMENTALS

2.1 Python Character Set :

It is a set of valid characters that a language recognize.

Letters: A-Z, a-z

Digits : 0-9

Special Symbols

Whitespace

2.2 TOKENS

Token: Smallest individual unit in a program is known as token.

There are five types of token in python:

1. Keyword
2. Identifier
3. Literal
4. Operators
5. Punctuators

1. **Keyword:** Reserved words in the library of a language. There are 33 keywords in python.

False	class	finally	is	return	break
None	continue	for	lambda	try	except
True	def	from	nonlocal	while	in
and	del	global	not	with	raise
as	elif	if	or	yield	
assert	else	import	pass		

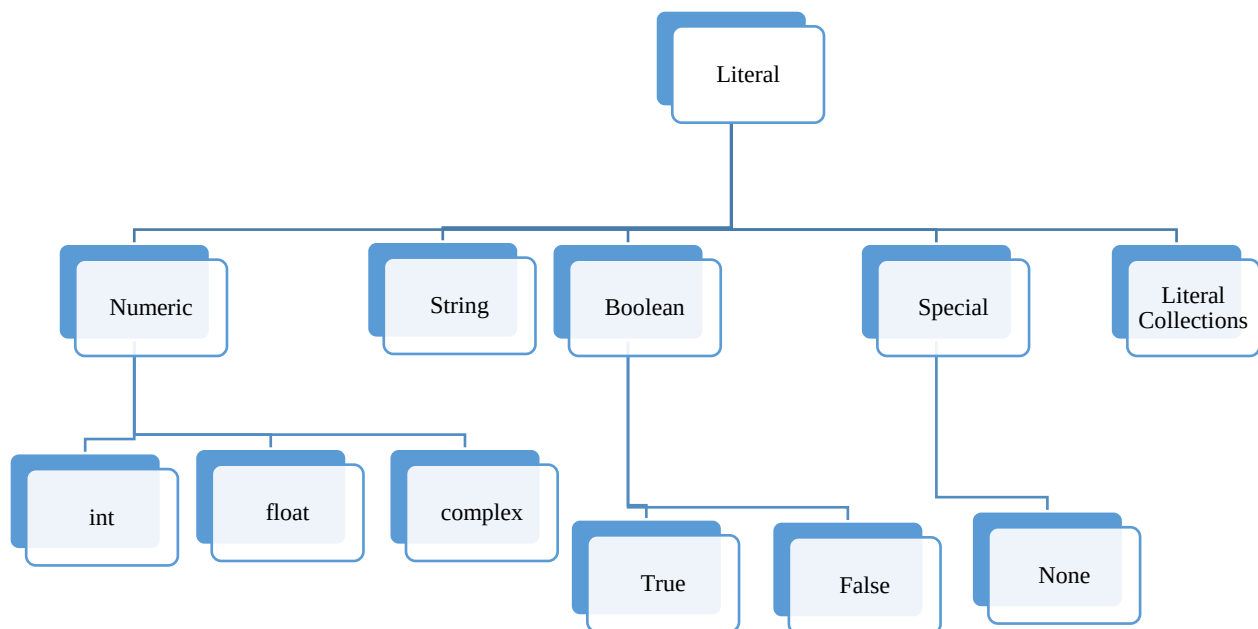
All the keywords are in lowercase except 03 keywords (True, False, None).

2. **Identifier:** The name given by the user to the entities like variable name, class-name, function-name etc.

Rules for identifiers:

- It can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore.
- It cannot start with a digit.
- Keywords cannot be used as an identifier.
- We cannot use special symbols like !, @, #, \$, %, + etc. in identifier.
- _ (underscore) can be used in identifier.
- Commas or blank spaces are not allowed within an identifier.

3. **Literal:** Literals are the constant value. Literals can be defined as a data that is given in a variable or constant.



A. Numeric literals: Numeric Literals are immutable.

Eg.

5, 6.7, 6+9j

B. String literals:

String literals can be formed by enclosing a text in the quotes. We can use both single as well as double quotes for a String.

Eg:

"Aman" , '12345'

Escape sequence characters:

\\	Backslash
\'	Single quote
\"	Double quote
\a	ASCII Bell
\b	Backspace
\f	ASCII Formfeed
\n	New line character
\t	Horizontal tab

C. Boolean literal: A Boolean literal can have any of the two values: **True** or **False**.

D. Special literals: Python contains one special literal i.e. **None**.

None is used to specify to that field that is not created. It is also used for end of lists in Python.

E. Literal Collections: Collections such as tuples, lists and Dictionary are used in Python.

4. Operators: An operator performs the operation on operands. Basically there are two types of operators in python according to number of operands:

A. Unary Operator

B. Binary Operator

A. Unary Operator: Performs the operation on one operand.

Example:

- + Unary plus
- Unary minus
- ~ Bitwise complement
- not Logical negation

B. Binary Operator: Performs operation on two operands.

5. Separator or punctuator : , ; , (), { }, []

2.3 Mantissa and Exponent Form:

A real number in exponent form has two parts:

- mantissa
- exponent

Mantissa : It must be either an integer or a proper real constant.

Exponent : It must be an integer. Represented by a letter E or e followed by integer value.

Valid Exponent form

123E05
1.23E07
0.123E08
123.0E08
123E+8
1230E04
-0.123E-3
163.E4
.34E-2
4.E3

Invalid Exponent form

2.3E (No digit specified for exponent)
0.24E3.2 (Exponent cannot have fractional part)
23,455E03 (No comma allowed)

2.4 Basic terms of a Python Programs:

- A. Blocks and Indentation
- B. Statements
- C. Expressions
- D. Comments

A. Blocks and Indentation:

- Python provides no braces to indicate blocks of code for class and function definition or flow control.
- Maximum line length should be maximum 79 characters.
- Blocks of code are denoted by line indentation, which is rigidly enforced.
- The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount.

for example –

if True:

 print("True")

else:

 print("False")

B. Statements

A line which has the instructions or expressions.

C. Expressions:

A legal combination of symbols and values that produce a result. Generally it produces a value.

D. Comments: Comments are not executed. Comments explain a program and make a program understandable and readable. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

There are two types of comments in python:

- i. Single line comment
- ii. Multi-line comment

i. **Single line comment:** This type of comments start in a line and when a line ends, it is automatically ends. Single line comment starts with # symbol.

Example: if a>b: # Relational operator compare two values

ii. **Multi-Line comment:** Multiline comments can be written in more than one lines. Triple quoted ' ' ' or " " " multi-line comments may be used in python. It is also known as **docstring**.

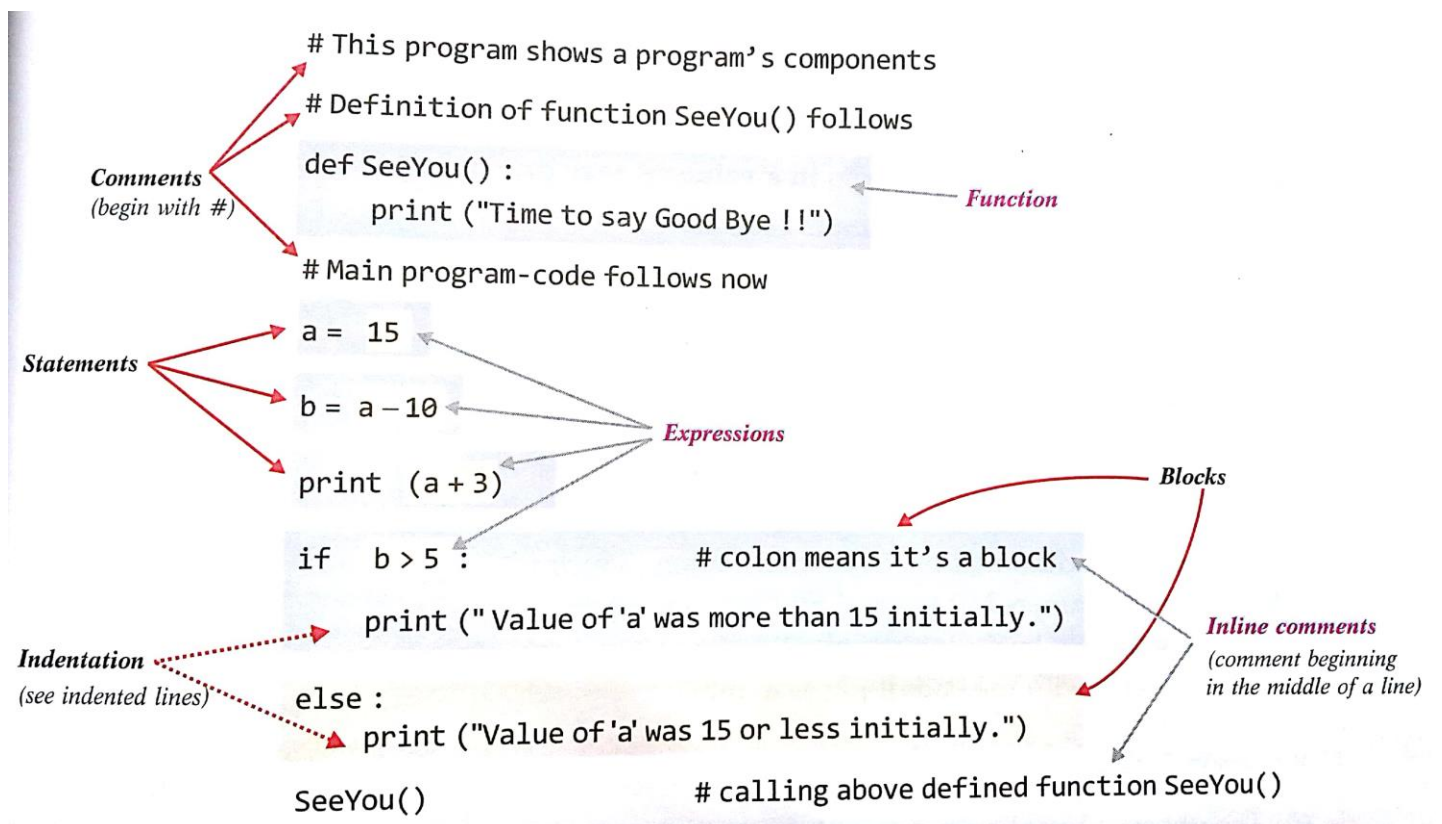
Example:

```
''' This program will calculate the average of 10 values.
```

```
    First find the sum of 10 values
```

```
    and divide the sum by number of values
```

```
'''
```



Multiple Statements on a Single Line:

The semicolon (;) allows multiple statements on the single line given that neither statement starts a new code block.

Example:-

```
x=5; print("Value =" x)
```

2.5 Variable/Label in Python:

Definition: Named location that refers to a value and whose value can be used and processed during program execution.

Variables in python do not have fixed locations. The location they refer to changes every time their values change.

Creating a variable:

A variable is created the moment you first assign a value to it.

Example:

```
x = 5
```

```
y = "hello"
```

Variables do not need to be declared with any particular type and can even change type after they have been set. It is known as dynamic Typing.

```
x = 4 # x is of type int
x = "python" # x is now of type str
print(x)
```

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscore (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Python allows assign a single value to multiple variables.

Example: `x = y = z = 5`

You can also assign multiple values to multiple variables. For example –

`x , y , z = 4, 5, "python"`

4 is assigned to x, 5 is assigned to y and string “python” assigned to variable z respectively.

`x=12`

`y=14`

`x,y=y,x`

`print(x,y)`

Now the result will be

14 12

Lvalue and Rvalue:

An expression has two values. Lvalue and Rvalue.

Lvalue: the LHS part of the expression

Rvalue: the RHS part of the expression

Python first evaluates the RHS expression and then assigns to LHS.

Example:

`p, q, r = 5, 10, 7`

`q, r, p = p+1, q+2, r-1`

`print (p,q,r)`

Now the result will be:

6 6 12

Note: Expressions separated with commas are evaluated from left to right and assigned in same order.

❖ If you want to know the type of variable, you can use **type()** function :

Syntax:

type (variable-name)

Example:

```
x=6
```

```
type(x)
```

The result will be:

```
<class 'int'>
```

- ❖ If you want to know the memory address or location of the object, you can use **id()** function.

Example:

```
>>>id(5)
```

```
1561184448
```

```
>>>b=5
```

```
>>>id(b)
```

```
1561184448
```

You can delete single or multiple variables by using del statement. Example:

```
del x
```

```
del y, z
```

2.6 Input from a user:

input() method is used to take input from the user.

Example:

```
print("Enter your name:")
```

```
x = input( )
```

```
print("Hello, " + x)
```

- input() function always returns a value of string type.

2.7 Type Casting:

To convert one data type into another data type.

Casting in python is therefore done using constructor functions:

- **int()** - constructs an integer number from an integer literal, a float literal or a string literal.

Example:

```
x = int(1) # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3
```

- **float()** - constructs a float number from an integer literal, a float literal or a string literal.

Example:

```
x = float(1) # x will be 1.0
y = float(2.8) # y will be 2.8
z = float("3") # z will be 3.0
w = float("4.2") # w will be 4.2
```

- **str()** - constructs a string from a wide variety of data types, including strings, integer literals and float literals.

Example:

```
x = str("s1") # x will be 's1'
y = str(2) # y will be '2'
z = str(3.0) # z will be '3.0'
```

Reading a number from a user:

```
x= int (input("Enter an integer number"))
```

2.8 OUTPUT using print() statement:

Syntax:

```
print(object, sep=<separator string >, end=<end-string>)
```

object : It can be one or multiple objects separated by comma.

sep : sep argument specifies the separator character or string. It separate the objects/items. By default sep argument adds space in between the items when printing.

end : It determines the end character that will be printed at the end of print line. By default it has newline character('\n').

Example:

```
x=10
```

```
y=20
```

```
z=30
```

```
print(x,y,z, sep='@', end= ' ')
```

Output:

```
10@20@30
```

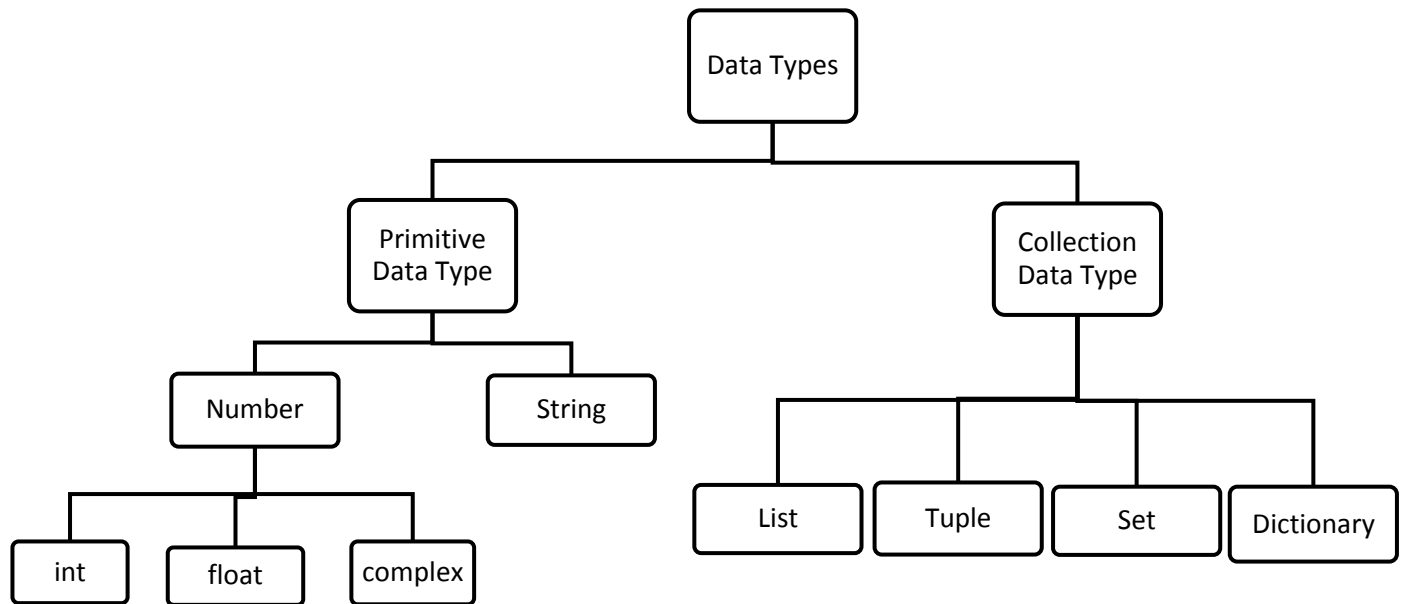
CHAPTER-3

DATA HANDLING

3.1 Data Types in Python:

Python has Two data types –

1. Primitive Data Type (Numbers, String)
2. Collection Data Type (List, Tuple, Set, Dictionary)



1. Primitive Data Types:

a. Numbers: Number data types store numeric values.

There are three numeric types in Python:

- int
- float
- complex

Example:

```
w = 1          # int
y = 2.8        # float
z = 1j         # complex
```


- **integer** : There are two types of integers in python:

- int
- Boolean

- **int**: int or integer, is a whole number, positive or negative, without decimals.

Example:

```
x = 1
y = 35656222554887711
z = -3255522
```

- **Boolean**: It has two values: True and False. **True** has the value **1** and **False** has the value **0**.

Example:

```
>>>bool(0)
```

False

```
>>>bool(1)
```

True

```
>>>bool('')
```

False

```
>>>bool(-34)
```

True

```
>>>bool(34)
```

True

- **float** : float or "floating point number" is a number, positive or negative, containing one or more decimals. Float can also be scientific numbers with an "e" to indicate the power of 10.

Example:

```
x = 1.10
y = 1.0
z = -35.59
```

```
a = 35e3
b = 12E4
c = -87.7e100
```

- **complex** : Complex numbers are written with a "j" as the imaginary part.

Example:

```
>>>x = 3+5j
```

```
>>>y = 2+4j
```

```
>>>z=x+y
```

```
>>>print(z)
```

```
5+9j
```

```
>>>z.real
```

```
5.0
```

```
>>>z.imag
```

```
9.0
```

Real and imaginary part of a number can be accessed through the attributes **real** and **imag**.

b. String: Sequence of characters represented in the quotation marks.

- Python allows for either pairs of single or double quotes. Example: 'hello' is the same as "hello" .
- Python does not have a character data type, a single character is simply a string with a length of 1.
- The python string store Unicode characters.
- Each character in a string has its own index.
- String is immutable data type means it can never change its value in place.

2. Collection Data Type:

- List
- Tuple
- Set
- Dictionary

3.2 MUTABLE & IMMUTABLE Data Type:

➤ Mutable Data Type:

These are changeable. In the same memory address, new value can be stored.

Example: List, Set, Dictionary

➤ Immutable Data Type:

These are unchangeable. In the same memory address new value cannot be stored.

Example: integer, float, Boolean, string and tuple.

3.3 Basic Operators in Python:

- i. Arithmetic Operators
- ii. Relational Operator
- iii. Logical Operators
- iv. Bitwise operators
- v. Assignment Operators
- vi. Other Special Operators
 - Identity Operators
 - Membership operators

i. **Arithmetic Operators:** To perform mathematical operations.

OPERATOR	NAME	SYNTAX	RESULT (X=14, Y=4)
+	Addition	$x + y$	18
-	Subtraction	$x - y$	10
*	Multiplication	$x * y$	56
/	Division (float)	x / y	3.5
//	Division (floor)	$x // y$	3
%	Modulus	$x \% y$	2
**	Exponent	$x ** y$	38416

Example:

```
>>>x= -5
>>>x**2
>>> -25
```

- ii. **Relational Operators:** Relational operators compare the values. It either returns **True** or **False** according to the condition.

OPERATOR	NAME	SYNTAX	RESULT (IF X=16, Y=42)
>	Greater than	$x > y$	False
<	Less than	$x < y$	True
==	Equal to	$x == y$	False
!=	Not equal to	$x != y$	True
>=	Greater than or equal to	$x >= y$	False
<=	Less than or equal to	$x <= y$	True

- iii. **Logical operators:** Logical operators perform **Logical AND**, **Logical OR** and **Logical NOT** operations.

OPERATOR	DESCRIPTION	SYNTAX
and	Logical AND: True if both the operands are true	x and y
or	Logical OR: True if either of the operands is true	x or y
not	Logical NOT: True if operand is false	not x

Examples of Logical Operator:

The **and** operator: The and operator works in two ways:

- Relational expressions as operands
- numbers or strings or lists as operands

a. Relational expressions as operands:

X	Y	X and Y
False	False	False
False	True	False
True	False	False
True	True	True

```
>>> 5>8 and 7>3
False
>>> (4==4) and (7==7)
True
```

b. numbers or strings or lists as operands:

In an expression X and Y, if first operand has **false value**, then return first operand X as a result, otherwise returns Y.

```
>>>0 and 0
```

```
0
```

```
>>>0 and 6
```

```
0
```

```
>>>'a' and 'n'
```

```
'n'
```

```
>>>6>9 and 'c'+9>5 # and operator will test the second operand only if the first operand
False              # is true, otherwise ignores it, even if the second operand is wrong
```

X	Y	X and Y
false	false	X
false	true	X
true	false	Y
true	true	Y

The **or** operator: The or operator works in two ways:

a. Relational expressions as operands

b. numbers or strings or lists as operands

a. Relational expressions as operands:

X	Y	X or Y
False	False	False
False	True	True
True	False	True
True	True	True

```
>>> 5>8 or 7>3
True
>>> (4==4) or (7==7)
True
```

b. numbers or strings or lists as operands:

In an expression X or Y, if first operand has **true value**, then return first operand X as a result, otherwise returns Y.

X	Y	X or Y
false	false	Y
false	true	Y
true	false	X
true	true	X

```
>>>0 or 0
```

```
0
```

```
>>>0 or 6
```

```
6
```

```
>>>'a' or 'n'
```

```
'a'
```

```
>>>6<9 or 'c'+9>5 # or operator will test the second operand only if the first operand
```

```
True # is false, otherwise ignores it, even if the second operand is wrong
```

The **not** operator:

```
>>>not 6
```

```
False
```

```
>>>not 0
```

```
True
```

```
>>>not -7
```

```
False
```

Chained Comparison Operators:

```
>>> 4<5>3 is equivalent to >>> 4<5 and 5>3
```

```
True True
```

iv. **Bitwise operators:** Bitwise operators acts on bits and performs bit by bit operation.

OPERATOR	DESCRIPTION	SYNTAX
&	Bitwise AND	x & y
	Bitwise OR	x y

~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

Examples:

Let

a = 10

b = 4

print(a & b)

print(a | b)

print(~a)

print(a ^ b)

print(a >> 2)

print(a << 2)

Output:

0

14

-11

14

2

40

v. **Assignment operators:** Assignment operators are used to assign values to the variables.

OPERA TOR	DESCRIPTION	SYNTAX
=	Assign value of right side of expression to left side operand	x = y + z
+=	Add AND: Add right side operand with left side operand and then assign to left operand	a+=b a=a+b
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand	a-=b a=a- b
=	Multiply AND: Multiply right operand with left operand and then assign to left operand	a=b a=a*b

<code>/=</code>	Divide AND: Divide left operand with right operand and then assign to left operand	<code>a/=b</code> <code>a=a/b</code>
<code>%=</code>	Modulus AND: Takes modulus using left and right operands and assign result to left operand	<code>a%=b</code> <code>a=a%b</code>
<code>//=</code>	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	<code>a//=b</code> <code>a=a//b</code>
<code>**=</code>	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	<code>a**=b</code> <code>a=a**b</code>
<code>&=</code>	Performs Bitwise AND on operands and assign value to left operand	<code>a&=b</code> <code>a=a&b</code>
<code> =</code>	Performs Bitwise OR on operands and assign value to left operand	<code>a =b</code> <code>a=a b</code>
<code>^=</code>	Performs Bitwise xOR on operands and assign value to left operand	<code>a^=b</code> <code>a=a^b</code>
<code>>>=</code>	Performs Bitwise right shift on operands and assign value to left operand	<code>a>>=b</code> <code>a=a>>b</code>
<code><<=</code>	Performs Bitwise left shift on operands and assign value to left operand	<code>a<<=b</code> <code>a=</code> <code>a<<b</code>

vi. **Other Special operators:** There are some special type of operators like-

- a. **Identity operators-** **is** and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

is	True if the operands are identical
is not	True if the operands are not identical

Example:

Let

`a1 = 3`

`b1 = 3`

`a2 = 'PythonProgramming'`

`b2 = 'PythonProgramming'`

`a3 = [1,2,3]`

`b3 = [1,2,3]`

`print(a1 is not b1)`


```
print(a2 is b2)      # Output is False, since lists are mutable.
print(a3 is b3)
```

Output:

False

True

False

Example:

```
>>>str1= "Hello"
```

```
>>>str2=input("Enter a String :")
```

Enter a String : Hello

```
>>>str1==str2      # compares values of string
```

True

```
>>>str1 is str2     # checks if two address refer to the same memory address
```

False

- b. **Membership operators-** **in** and **not in** are the membership operators; used to test whether a value or variable is in a sequence.

in	True if value is found in the sequence
not in	True if value is not found in the sequence

Example:

Let

```
x = 'Digital India'
```

```
y = {3:'a',4:'b'}
```

```
print('D' in x)
```

```
print('digital' not in x)
```

```
print('Digital' not in x)
```

```
print(3 in y)
```

```
print('b' in y)
```

Output:

True

True

False

True

False

3.4 Operator Precedence and Associativity:

Operator Precedence: It describes the order in which operations are performed when an expression is evaluated. Operators with higher **precedence** perform the operation first.

Operator Associativity: whenever two or more operators have the same precedence, then associativity defines the order of operations.

Operator	Description	Associativity	Precedence
(), { }	Parentheses (grouping)	Left to Right	
f(args...)	Function call	Left to Right	
x[index:index]	Slicing	Left to Right	
x[index]	Subscription	Left to Right	
**	Exponent	Right to Left	
~x	Bitwise not	Left to Right	
+x, -x	Positive, negative	Left to Right	
*, /, %	Product, division, remainder	Left to Right	
+, -	Addition, subtraction	Left to Right	
<<, >>	Shifts left/right	Left to Right	
&	Bitwise AND	Left to Right	
^	Bitwise XOR	Left to Right	
	Bitwise OR	Left to Right	
<=, <, >, >=	Comparisons	Left to Right	
=, %=, /=, +=	Assignment		
is, is not	Identity		
in, not in	Membership		
not	Boolean NOT	Left to Right	
and	Boolean AND	Left to Right	
or	Boolean OR	Left to Right	
lambda	Lambda expression	Left to Right	

CHAPTER-4

FLOW OF CONTROL

1. Decision Making and branching (Conditional Statement)

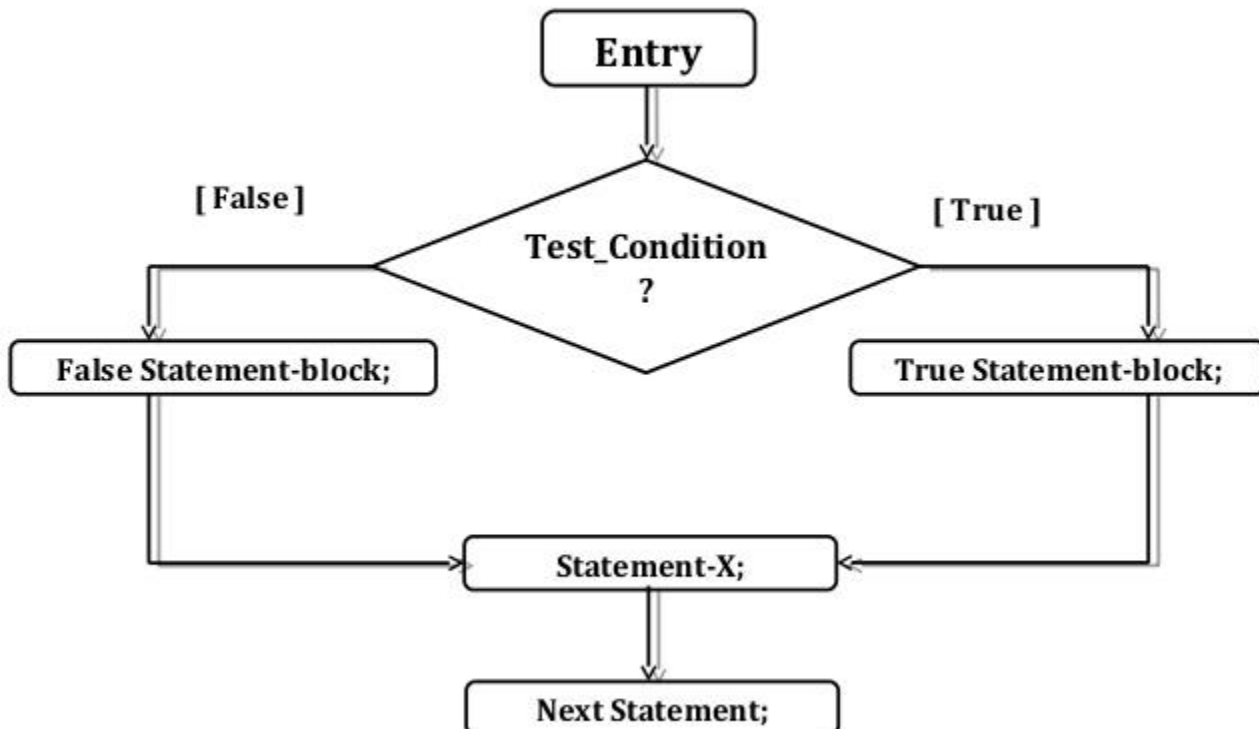
2. Looping or Iteration

3. Jumping statements

4.1 DECISION MAKING & BRANCHING

Decision making is about deciding the order of execution of statements based on certain conditions. Decision structures evaluate multiple expressions which produce TRUE or FALSE as outcome.

if else Statement- Flowchart



There are three types of conditions in python:

1. if statement
2. if-else statement
3. elif statement

1. if statement: It is a simple if statement. When condition is true, then code which is associated with if statement will execute.

Example:

```
a=40
b=20
if a>b:
    print("a is greater than b")
```

2. if-else statement: When the condition is true, then code associated with if statement will execute, otherwise code associated with else statement will execute.

Example:

```
a=10
b=20
if a>b:
    print("a is greater")
else:
    print("b is greater")
```

3. elif statement: It is short form of else-if statement. If the previous conditions were not true, then do this condition". It is also known as nested if statement.

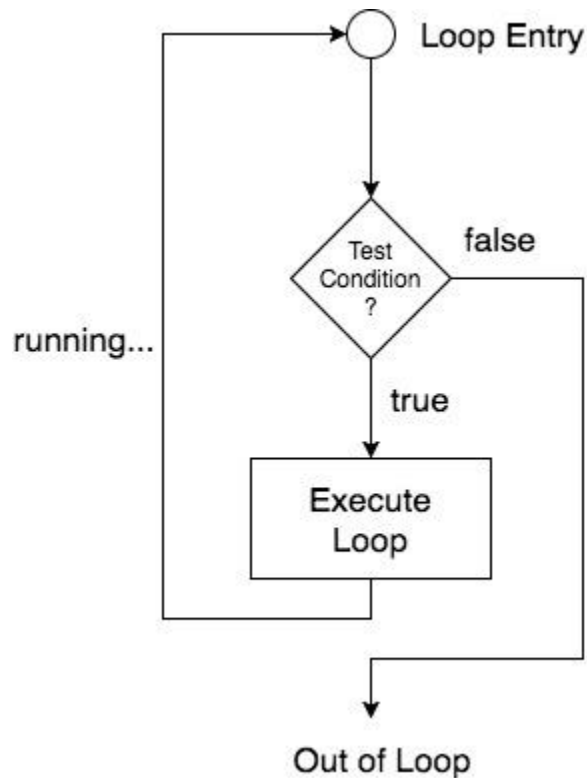
Example:

```
a=input("Enter first number")
b=input("Enter Second Number:")
if a>b:
```

```
print("a is greater")
elif a==b:
    print("both numbers are equal")
else:
    print("b is greater")
```

4.2 LOOPS in PYTHON

Loop: Execute a set of statements repeatedly until a particular condition is satisfied.



There are two types of loops in python:

1. while loop
2. for loop

